- bot: a subtype of all RBS types.
- class: equivalent to the singleton class of the class being defined.
- instance: the type of an instance of the class being defined.
- nil: means nil.
- self: indicates the type of the receiver of the method.
- top: a supertype of all RBS types, very similar to untyped.
- untyped: RBS does not care what this variable is. The equivalent of TypeScript's any.
- void: equivalent to top. Use in cases where the return value of a method will not be used.

RBS Generics

A common problem in a type system is a container that behaves similarly no matter what type of objects are inside it. So, an array of String has a lot of methods that return a String, while an array of Integer has all the same methods, but they return an Integer. The container doesn't care what type the values are, but it does care that the values are consistent for any particular container.

The term for this in typed languages is a *generic*. Typically, the container class or module or interface defines one or more generic types as being part of the container, and then methods within that container use those generics to represent the type as a parameter or return type.

In RBS, a class, interface, or module can define one or more generic types in square brackets at the top of the definition. The type names are capitalized, and by convention, they are usually a single letter.

In this case the Scheduler class defines a generic, the next method returns an element of that type, and the perform method takes an element of that type as an argument:

```
class Scheduler[T]
  def next: () -> T
  def perform: (T) -> void
end
```

When you use Scheduler in other RBS definitions, you augment it with the type to be used as a generic. So Scheduler[RedisJob] would imply that next will return an object of type RedisJob and that perform will take an object of type RedisJob.

You might want to limit the input type going to the generic. You can specify a limiting parent class to the generic:

```
class Scheduler[T < Job]
  def next: () -> T
  def perform: (T) -> void
end
```

So now when Scheudler[RedisJob] is declared, RBS ensures that RedisJob is a subclass of Job.

By default, the contained classes much match with their usages—if you declare that something is of type Scheduler[RedisJob] and then try to pass that variable to something that expects Scheduler[Job], RBS will consider that to be a type mismatch, even though Job is a superclass of RedisJob.

In order to make this work, you can describe the scheduler generic as [out T]. If so described, then Scheduler instances will type match with generics that are superclasses of the generic.